МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №5

з дисципліни «Алгоритми та структури даних»

Виконав:

студент групи ІМ-о51

Машута О.І

Перевірив:

ас. Череватенко О.В.

Київ 2026

Постановка задачі 1. Представити напрямлений граф із заданими параметрами так само, як у лабораторній роботі №3. Відмінність:

коефіцієнт k 1.0 n3 0.01 n4 0.005 0.15. Отже, матриця суміжності Adir напрямленого графа за варіантом фор мується таким чином: 1) встановлюється параметр (seed) генератора випадкових чисел, рів не номеру варіанту n1n2n3n4; 2) матриця розміром n n заповнюється згенерованими випадковими числами в діапазоні [0, 2.0); 3) обчислюється коефіцієнт k 1.0 n3 0.01 n4 0.005 0.15, кожен елемент матриці множиться на коефіцієнт k; 4) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0. 2. Створити програму, яка виконує обхід напрямленого графа вшир (BFS) та вглиб (DFS). • обхід починати з вершини із найменшим номером, яка має щонай менше одну вихідну дугу; • при обході враховувати порядок нумерації; 46 • у програмі виконання обходу відображати покроково, черговий крок виконувати за натисканням кнопки у вікні або на клавіатурі. 3. Під час обходу графа побудувати дерево обходу. У програмі дерево об ходу виводити покроково у процесі виконання обходу графа. Це можна виконати одним із двох способів: • або виділяти іншим кольором ребра графа; • або будувати дерево обходу поряд із графом. 4. Зміну статусів вершин у процесі обходу продемонструвати зміною ко льорів вершин, графічними позначками тощо, або ж у процесі обходу виводити протокол обходу у графічне вікно або в консоль. 5. Якщо після обходу графа лишилися невідвідані вершини, продовжува ти обхід з невідвіданої вершини з найменшим номером, яка має щонай менше одну вихідну дугу. При проєктуванні програми також слід врахувати наступне: 1) мова програмування обирається студентом самостійно; 2) графічне зображення усіх графів має формуватися програмою з тими жвимогами, як у ЛР №3; 3) всі графи обов’язково зображувати у графічному вікні; 4) типи та структури даних для внутрішнього представлення всіх даних у програмі слід вибрати самостійно

Текст програми: using System; using System.Collections.Generic; using System.Drawing; using System.Drawing.Drawing2D; using System.Linq; using System.Windows.Forms;

namespace GraphLab5 { static class Program { [STAThread] static void Main() { Application.SetHighDpiMode(HighDpiMode.SystemAware); Application.EnableVisualStyles(); Application.SetCompatibleTextRenderingDefault(false); Application.Run(new Form1()); } }

```
public class TraversalStep
{
    public HashSet<int> Visited { get; set; } = new
HashSet<int>();
    public List<Tuple<int, int>> TreeEdges { get; set; } =
new List<Tuple<int, int>>();
    public int CurrentNode { get; set; } = -1;
    public string LogMessage { get; set; } = "";

    public TraversalStep Clone()
    {
        return new TraversalStep
        {
            Visited = new HashSet<int>(this.Visited),
            TreeEdges = new List<Tuple<int,
int>>(this.TreeEdges),
            CurrentNode = this.CurrentNode,
            LogMessage = this.LogMessage
        };
    }
}

public partial class Form1 : Form
{
    const int n1 = 3;
    const int n2 = 4;
    const int n3 = 1;
    const int n4 = 2;

    private int N;
    private int[,] matrix;
```

```
    private List<TraversalStep> steps = new
List<TraversalStep>();
    private int currentStepIndex = -1;

    private PictureBox pbGraph;
    private RichTextBox txtLog;
    private Button btnGen, btnBFS, btnDFS, btnNext;
    private Label lblStatus;

    public Form1()
    {
        SetupUI();
        N = 10 + n3;
    }

    private void GenerateGraph(object sender, EventArgs e)
    {
        steps.Clear();
        currentStepIndex = -1;
        txtLog.Clear();
        lblStatus.Text = "Graph generated. Choose a
traversal method.";
        btnNext.Enabled = false;

        int seed = int.Parse($"{n1}{n2}{n3}{n4}");
        Random rand = new Random(seed);
        matrix = new int[N, N];

        double k = 1.0 - (n3 * 0.01) - (n4 * 0.005) -
0.15;
        txtLog.AppendText($"Generating
matrix...\nCoefficient K = {k:F3}\n");

        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
```

```
            {
                double val = rand.NextDouble() * 2.0;
                val *= k;
                matrix[i, j] = (val >= 1.0) ? 1 : 0;
            }
        }

        PrintMatrix(matrix);
        pbGraph.Invalidate();
    }

    private int GetStartNode(HashSet<int> visitedGlobal)
    {
        for (int i = 0; i < N; i++)
        {
            if (!visitedGlobal.Contains(i))
            {
                int outDegree = 0;
                for (int j = 0; j < N; j++) outDegree +=
matrix[i, j];

                if (outDegree > 0) return i;
            }
        }
        return -1;
    }

    private void StartBFS(object sender, EventArgs e)
    {
        PrepareTraversal("BFS (Breadth-first)");

        HashSet<int> visitedGlobal = new HashSet<int>();
        TraversalStep currentStep = new TraversalStep();

        while (true)
        {
            int startNode = GetStartNode(visitedGlobal);
```

```
            if (startNode == -1) break;

            Queue<int> q = new Queue<int>();
            q.Enqueue(startNode);
            visitedGlobal.Add(startNode);

            currentStep = currentStep.Clone();
            currentStep.Visited.Add(startNode);
            currentStep.CurrentNode = startNode;
            currentStep.LogMessage = $"Start new component
at vertex {startNode + 1}";
            steps.Add(currentStep);

            while (q.Count > 0)
            {
                int u = q.Dequeue();

                currentStep = currentStep.Clone();
                currentStep.CurrentNode = u;
                currentStep.LogMessage = $"Processing
vertex {u + 1}";
                steps.Add(currentStep);

                for (int v = 0; v < N; v++)
                {
                    if (matrix[u, v] == 1 &&
!visitedGlobal.Contains(v))
                    {
                        visitedGlobal.Add(v);
                        q.Enqueue(v);

                        currentStep = currentStep.Clone();
                        currentStep.TreeEdges.Add(new
Tuple<int, int>(u, v));
                        currentStep.Visited.Add(v);
                        currentStep.LogMessage = $" ->
Move to {v + 1} (Tree)";
```

```
                    steps.Add(currentStep);
                }
            }
        }
    }
    FinalizeTraversal();
}

private void StartDFS(object sender, EventArgs e)
{
    PrepareTraversal("DFS (Depth-first)");

    HashSet<int> visitedGlobal = new HashSet<int>();
    TraversalStep currentStep = new TraversalStep();

    while (true)
    {
        int startNode = GetStartNode(visitedGlobal);
        if (startNode == -1) break;

        Stack<int> s = new Stack<int>();
        s.Push(startNode);

        int[] parent = new int[N];
        for (int i = 0; i < N; i++) parent[i] = -1;

        while (s.Count > 0)
        {
            int u = s.Pop();

            if (!visitedGlobal.Contains(u))
            {
                visitedGlobal.Add(u);

                currentStep = currentStep.Clone();
                currentStep.CurrentNode = u;
                currentStep.Visited.Add(u);
```

```
                    if (parent[u] != -1)
                    {
                        currentStep.TreeEdges.Add(new
Tuple<int, int>(parent[u], u));
                        currentStep.LogMessage = $" ->
Move to {u + 1} (from {parent[u] + 1})";
                    }
                    else
                    {
                        currentStep.LogMessage = $"Start
component from {u + 1}";
                    }
                    steps.Add(currentStep);

                    for (int v = N - 1; v >= 0; v--)
                    {
                        if (matrix[u, v] == 1 &&
!visitedGlobal.Contains(v))
                        {
                            s.Push(v);
                            parent[v] = u;
                        }
                    }
                }
            }
        }
        FinalizeTraversal();
    }

    private void PrepareTraversal(string name)
    {
        if (matrix == null) { MessageBox.Show("Generate
the graph first!"); return; }
        steps.Clear();
        currentStepIndex = -1;
        txtLog.Clear();
```

```
        txtLog.AppendText($"--- Start traversal {name} ---
\n");
        btnNext.Enabled = true;
        pbGraph.Invalidate();
    }

    private void FinalizeTraversal()
    {
        if (steps.Count == 0)
        {
            txtLog.AppendText("No vertices satisfy the
start condition (with outgoing edges).\n");
            btnNext.Enabled = false;
        }
        else
        {
            txtLog.AppendText($"Traversal planned. Steps:
{steps.Count}. Press 'Next step'.\n");
            currentStepIndex = -1;
        }
    }

    private void ShowNextStep(object sender, EventArgs e)
    {
        if (currentStepIndex < steps.Count - 1)
        {
            currentStepIndex++;
            var s = steps[currentStepIndex];
            txtLog.AppendText($"Step {currentStepIndex +
1}: {s.LogMessage}\n");
            txtLog.ScrollToCaret();
            lblStatus.Text = $"Step {currentStepIndex + 1}
/ {steps.Count}";
            pbGraph.Invalidate();
        }
        else
```

```
        {
            MessageBox.Show("Traversal complete!");
            btnNext.Enabled = false;
        }
    }

    private void pbGraph_Paint(object sender,
PaintEventArgs e)
    {
        e.Graphics.SmoothingMode =
SmoothingMode.AntiAlias;
        if (matrix == null) return;

        int w = pbGraph.Width;
        int h = pbGraph.Height;
        int cx = w / 2;
        int cy = h / 2;
        int r = Math.Min(w, h) / 2 - 40;

        PointF[] points = GetCircleCoords(N, cx, cy, r);

        TraversalStep state = (currentStepIndex >= 0 &&
currentStepIndex < steps.Count)
                                ? steps[currentStepIndex]
                                : new TraversalStep();

        Pen penNormal = new Pen(Color.LightGray, 1);
        penNormal.CustomEndCap = new
System.Drawing.Drawing2D.AdjustableArrowCap(4, 4);

        for (int i = 0; i < N; i++)
        {
            for (int j = 0; j < N; j++)
            {
                if (matrix[i, j] == 1)
                {
                    bool isTreeEdge =
```

```
state.TreeEdges.Any(t => t.Item1 == i && t.Item2 == j);
                    if (!isTreeEdge)
                    {
                        DrawArrow(e.Graphics, penNormal,
points[i], points[j]);
                    }
                }
            }
        }

        Pen penTree = new Pen(Color.Blue, 3);
        penTree.CustomEndCap = new
System.Drawing.Drawing2D.AdjustableArrowCap(5, 5);
        foreach (var edge in state.TreeEdges)
        {
            DrawArrow(e.Graphics, penTree,
points[edge.Item1], points[edge.Item2]);
        }

        for (int i = 0; i < N; i++)
        {
            Brush bg = Brushes.White;
            Pen border = Pens.Black;

            if (state.Visited.Contains(i)) bg =
Brushes.LightYellow;

            if (i == state.CurrentNode)
            {
                bg = Brushes.Coral;
                border = new Pen(Color.Red, 2);
            }

            DrawNode(e.Graphics, points[i], (i +
1).ToString(), bg, border);
        }
    }
```

```
    private void PrintMatrix(int[,] mat)
    {
        txtLog.AppendText("Adjacency matrix:\n");
        txtLog.AppendText("   ");
        for (int j = 0; j < N; j++) txtLog.AppendText($"{j
+ 1,2} ");
        txtLog.AppendText("\n");
        for (int i = 0; i < N; i++)
        {
            txtLog.AppendText($"{i + 1,2}|");
            for (int j = 0; j < N; j++)
txtLog.AppendText($"{mat[i, j],2} ");
            txtLog.AppendText("\n");
        }
    }

    private PointF[] GetCircleCoords(int count, int cx,
int cy, int r)
    {
        PointF[] pts = new PointF[count];
        double angleStep = 2 * Math.PI / count;
        for (int i = 0; i < count; i++)
        {
            pts[i] = new PointF(
                (float)(cx + r * Math.Cos(i * angleStep -
Math.PI / 2)),
                (float)(cy + r * Math.Sin(i * angleStep -
Math.PI / 2))
            );
        }
        return pts;
    }

    private void DrawNode(Graphics g, PointF p, string
label, Brush bg, Pen border)
    {
```

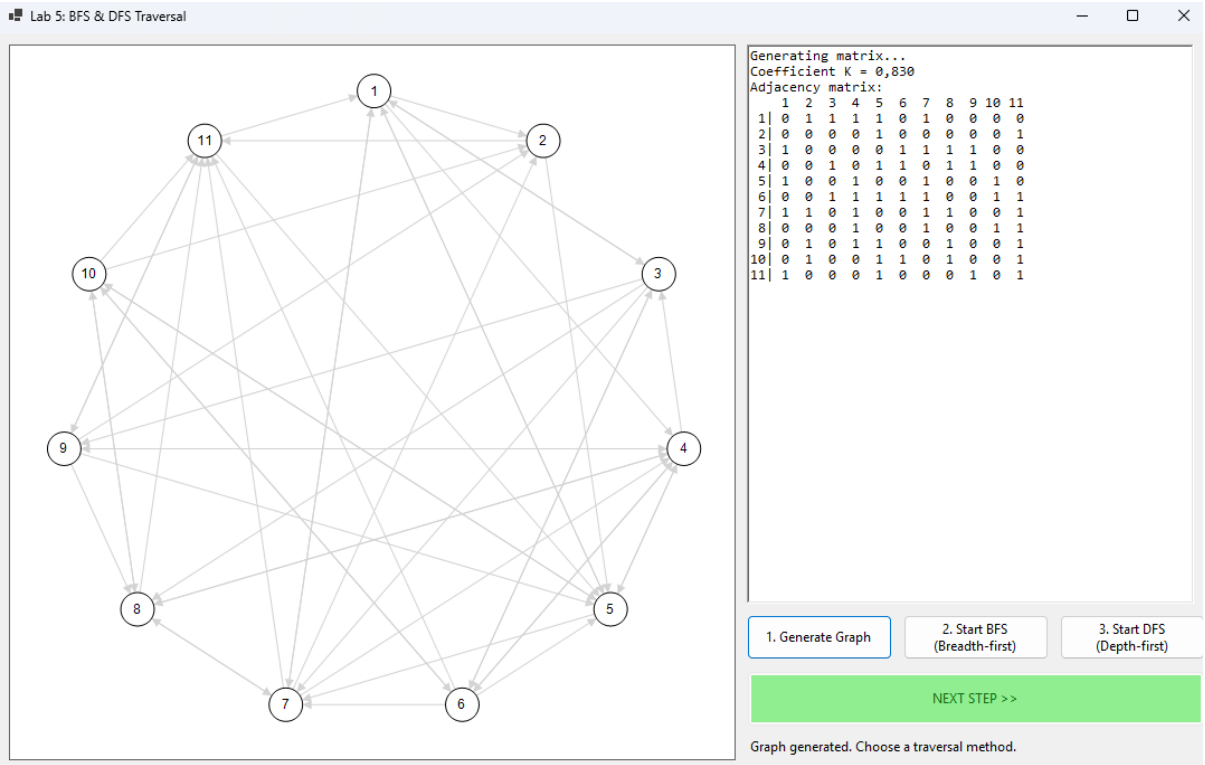
```
        float r = 15;
        RectangleF rect = new RectangleF(p.X - r, p.Y - r,
2 * r, 2 * r);
        g.FillEllipse(bg, rect);
        g.DrawEllipse(border, rect);

        StringFormat sf = new StringFormat() { Alignment =
StringAlignment.Center, LineAlignment =
StringAlignment.Center };
        g.DrawString(label, new Font("Arial", 9),
Brushes.Black, rect, sf);
    }

    private void DrawArrow(Graphics g, Pen p, PointF p1,
PointF p2)
    {
        float r = 15;
        double angle = Math.Atan2(p2.Y - p1.Y, p2.X -
p1.X);
        PointF start = new PointF((float)(p1.X + r *
Math.Cos(angle)), (float)(p1.Y + r * Math.Sin(angle)));
        PointF end = new PointF((float)(p2.X - r *
Math.Cos(angle)), (float)(p2.Y - r * Math.Sin(angle)));
        g.DrawLine(p, start, end);
    }

    private void SetupUI()
    {
        this.Size = new Size(1100, 700);
        this.Text = "Lab 5: BFS & DFS Traversal";

        pbGraph = new PictureBox();
        pbGraph.Location = new Point(10, 10);
        pbGraph.Size = new Size(650, 640);
        pbGraph.BorderStyle = BorderStyle.FixedSingle;
        pbGraph.BackColor = Color.White;
        pbGraph.Paint += pbGraph_Paint;
```

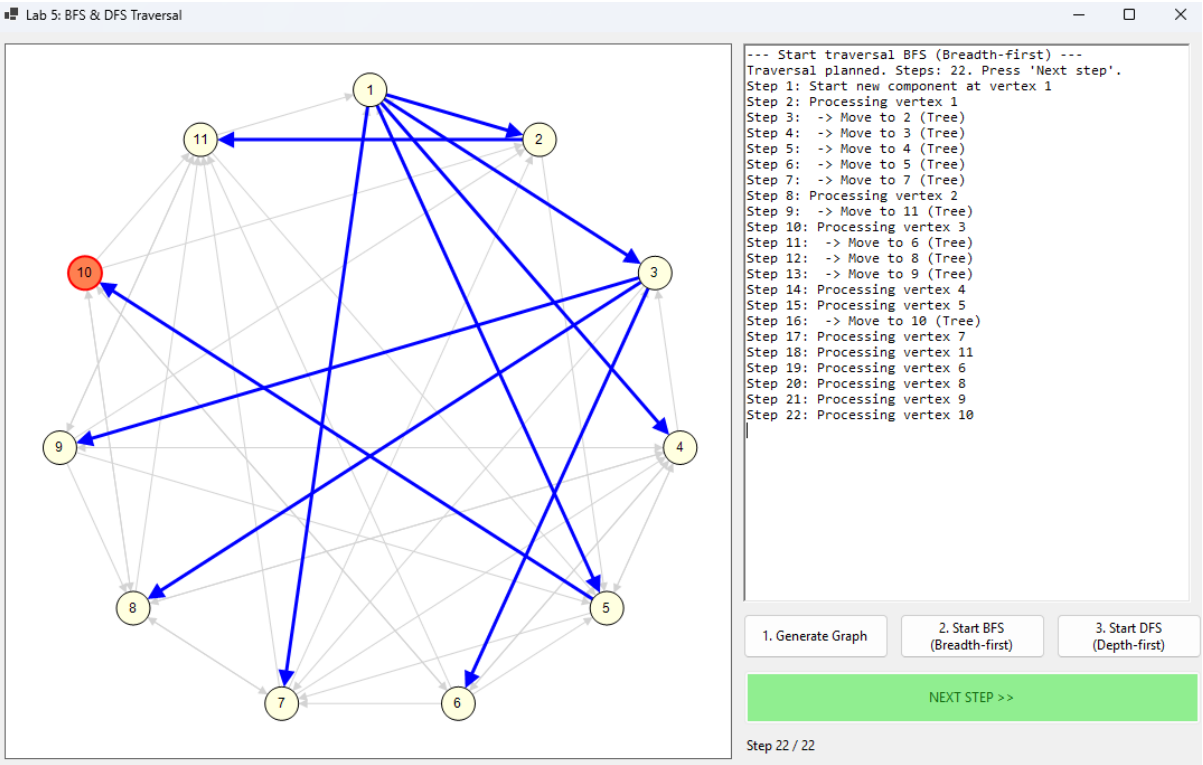
```
        this.Controls.Add(pbGraph);

        txtLog = new RichTextBox();
        txtLog.Location = new Point(670, 10);
        txtLog.Size = new Size(400, 500);
        txtLog.Font = new Font("Consolas", 9);
        this.Controls.Add(txtLog);

        btnGen = new Button() { Text = "1. Generate
Graph", Location = new Point(670, 520), Size = new
Size(130, 40) };
        btnGen.Click += GenerateGraph;
        this.Controls.Add(btnGen);

        btnBFS = new Button() { Text = "2. Start BFS
(Breadth-first)", Location = new Point(810, 520), Size =
new Size(130, 40) };
        btnBFS.Click += StartBFS;
        this.Controls.Add(btnBFS);

        btnDFS = new Button() { Text = "3. Start DFS
(Depth-first)", Location = new Point(950, 520), Size = new
Size(130, 40) };
        btnDFS.Click += StartDFS;
        this.Controls.Add(btnDFS);

        btnNext = new Button() { Text = "NEXT STEP >>",
Location = new Point(670, 570), Size = new Size(410, 50),
BackColor = Color.LightGreen, Enabled = false };
        btnNext.Click += ShowNextStep;
        this.Controls.Add(btnNext);

        lblStatus = new Label() { Location = new
Point(670, 630), AutoSize = true, Text = "Waiting..." };
        this.Controls.Add(lblStatus);
    }
```

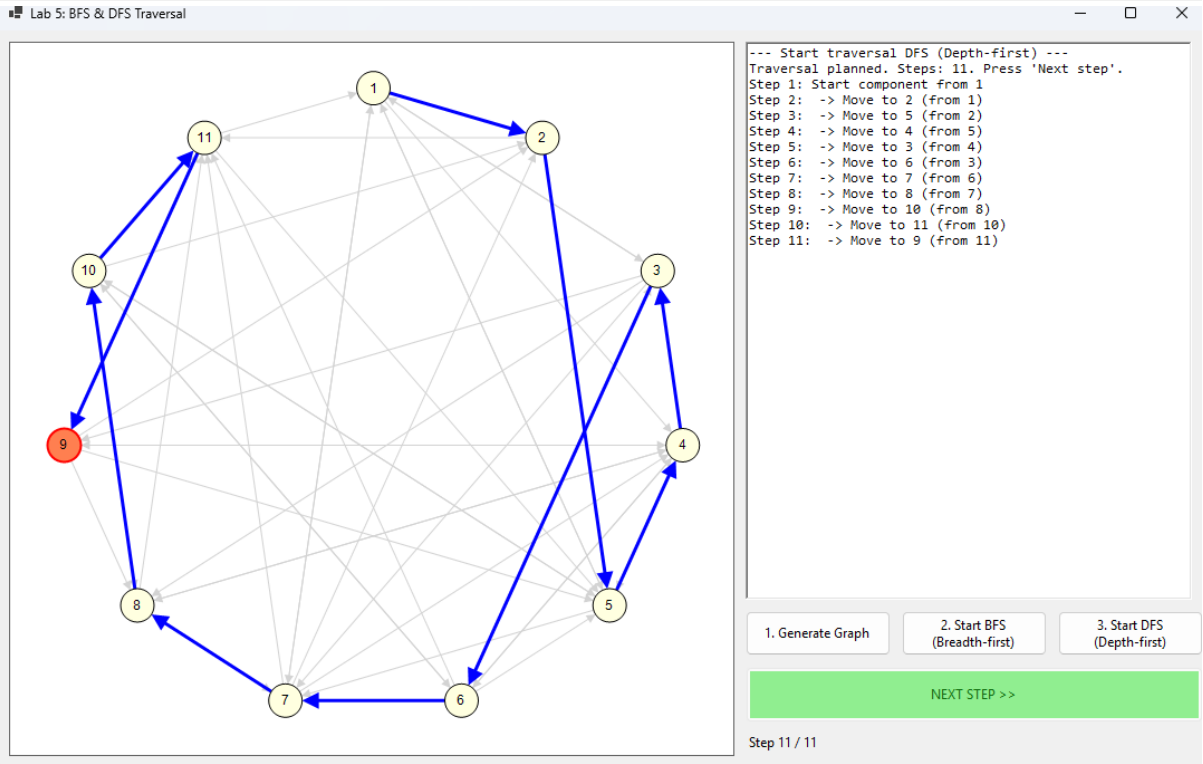
}

}

Тест програми







Висновок: У ході виконання лабораторної роботи №5 було вивчено та програмно реалізовано методи обходу графа в ширину (BFS) та в глибину (DFS). У процесі виконання роботи було побудовано дерева обходу, які наочно демонструють зв’язки та ієрархію вершин при послідовному дослідженні графа. Завдяки покроковій візуалізації статусів вершин вдалося простежити динаміку роботи алгоритмів, що дозволило краще зрозуміти механізми виявлення компонентів зв'язності та принципи формування остовних дерев. Реалізована програма

забезпечує автоматизований обхід згідно з заданими правилами нумерації, що підтверджує ефективність обраних структур даних для вирішення задач теорії графів.